Utilizing Convolutional Neural Networks and Computer Vision for Traffic Sign Recognition

Bacani, Cyril Cris T.

Magpantay, Sheena Marie B.

Egaran, Chelsea Felize B.

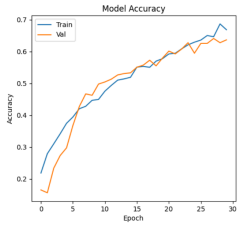
Manalo, Kisha Margarette B.

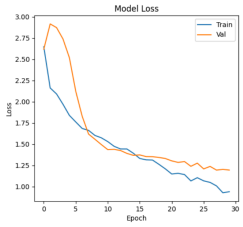
Checkpoint Report

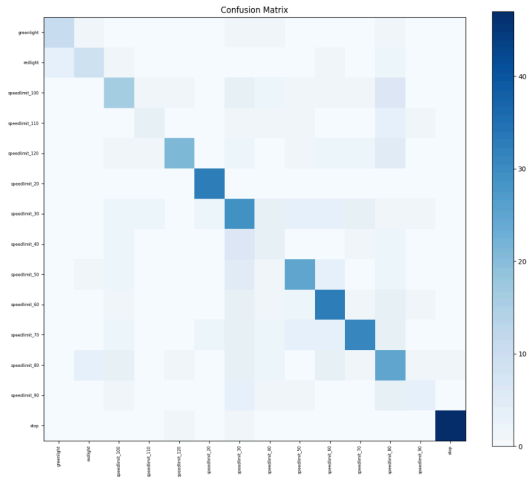
• Baselines trained (simple ML + DL); initial metrics logged.

traffic-sign-classifier-main > docs > screenshots > classification_report.txt

	precision	recall	f1-score	support
greenlight	0.79	0.73	0.76	15
redlight	0.64	0.56	0.60	16
speedlimit_100	0.50	0.48	0.49	33
speedlimit_110	0.50	0.36	0.42	11
speedlimit_120	0.88	0.60	0.71	35
speedlimit_20	0.89	1.00	0.94	33
speedlimit_30	0.45	0.57	0.50	51
speedlimit_40	0.21	0.27	0.24	15
speedlimit_50	0.68	0.64	0.66	39
speedlimit_60	0.66	0.70	0.68	47
speedlimit_70	0.76	0.61	0.67	51
speedlimit_80	0.42	0.54	0.48	46
speedlimit_90	0.43	0.23	0.30	13
stop	0.98	0.96	0.97	49
accuracy			0.64	454
macro avg	0.63	0.59	0.60	454
weighted avg	0.66	0.64	0.64	454







• CNN experiment running (first results); NLP component scaffolded/prototyped.

train.py | u.py | eval.py | rl_agent.py | cnn_model.py

C: > Users > MAGPANTAY > Downloads > train.py > ...

```python
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import keras
from keras import layers, models, optimizers, callbacks
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATADIR = os.path.join(BASE_DIR, "data", "dataset")
SCREENSHOT_DIR = os.path.join(BASE_DIR, "docs", "screenshots")

IMG_SIZE = 32
CHANNELS = 3
BATCH_SIZE = 32
EPOCHS = 30

if not os.path.exists(SCREENSHOT_DIR):
    os.makedirs(SCREENSHOT_DIR)

# ---------------------------
# DATA LOADING & PREPROCESSING
# ---------------------------
def load_and_preprocess_data():
    images = []
    labels = []

    if not os.path.exists(DATADIR):
        raise FileNotFoundError(f"Directory {DATADIR} not found at {DATADIR}")

    categories = sorted(os.listdir(DATADIR))
    label_map = {category: i for i, category in enumerate(categories)}

    print(f"Loading {len(categories)} classes...")

    for category, label_index in label_map.items():
        path = os.path.join(DATADIR, category)
        if not os.path.isdir(path): continue
        for img_name in os.listdir(path):
            try:
                img_path = os.path.join(path, img_name)
                img = cv2.imread(img_path)
                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
Epoch 10/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.4405 - loss: 1.5743 - val_accuracy: 0.4978 - val_loss: 1.4046 - learning_rate: 2.5000e-04
Epoch 11/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.4755 - loss: 1.5300 - val_accuracy: 0.5044 - val_loss: 1.4354 - learning_rate: 2.5000e-04
Epoch 12/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.4837 - loss: 1.4741 - val_accuracy: 0.5132 - val_loss: 1.4386 - learning_rate: 2.5000e-04
Epoch 13/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5102 - loss: 1.4430 - val_accuracy: 0.5264 - val_loss: 1.4247 - learning_rate: 2.5000e-04
Epoch 14/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5141 - loss: 1.4426 - val_accuracy: 0.5308 - val_loss: 1.3907 - learning_rate: 2.5000e-04
Epoch 15/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.5190 - loss: 1.3927 - val_accuracy: 0.5330 - val_loss: 1.3679 - learning_rate: 2.5000e-04
Epoch 16/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5510 - loss: 1.3315 - val_accuracy: 0.5587 - val_loss: 1.3729 - learning_rate: 2.5000e-04
Epoch 17/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5532 - loss: 1.3149 - val_accuracy: 0.5573 - val_loss: 1.3536 - learning_rate: 2.5000e-04
Epoch 18/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5505 - loss: 1.3123 - val_accuracy: 0.5727 - val_loss: 1.3517 - learning_rate: 2.5000e-04
Epoch 19/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5698 - loss: 1.2622 - val_accuracy: 0.5551 - val_loss: 1.3433 - learning_rate: 2.5000e-04
Epoch 20/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.5775 - loss: 1.2090 - val_accuracy: 0.5793 - val_loss: 1.3308 - learning_rate: 2.5000e-04
Epoch 21/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5924 - loss: 1.1483 - val_accuracy: 0.6013 - val_loss: 1.3028 - learning_rate: 2.5000e-04
Epoch 22/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5946 - loss: 1.1560 - val_accuracy: 0.5925 - val_loss: 1.2849 - learning_rate: 2.5000e-04
Epoch 23/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.6084 - loss: 1.1418 - val_accuracy: 0.6079 - val_loss: 1.2948 - learning_rate: 2.5000e-04
Epoch 24/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.6211 - loss: 1.0865 - val_accuracy: 0.6278 - val_loss: 1.2387 - learning_rate: 2.5000e-04
Epoch 25/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.6293 - loss: 1.1046 - val_accuracy: 0.5947 - val_loss: 1.2760 - learning_rate: 2.5000e-04
Epoch 26/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.6360 - loss: 1.0675 - val_accuracy: 0.6256 - val_loss: 1.2083 - learning_rate: 2.5000e-04
Epoch 27/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.6503 - loss: 1.0493 - val_accuracy: 0.6256 - val_loss: 1.2377 - learning_rate: 2.5000e-04
Epoch 28/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.6464 - loss: 1.0081 - val_accuracy: 0.6410 - val_loss: 1.1942 - learning_rate: 2.5000e-04
Epoch 29/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.6867 - loss: 0.9272 - val_accuracy: 0.6278 - val_loss: 1.2029 - learning_rate: 2.5000e-04
Epoch 30/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.6685 - loss: 0.9396 - val_accuracy: 0.6366 - val_loss: 1.1947 - learning_rate: 2.5000e-04

--- Generating Checkpoint Artifacts ---
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
Artifacts successfully saved to: c:\Users\Admin\Downloads\traffic-sign-classifier-main-20260321T060516Z-1-001\traffic-sign-classifier-main\docs\screenshots
```

• RL agent stubbed with reward design; early learning curves (even if noisy).

rl_agent.py

C: > Users > MAGPANTAY > Downloads > rl_agent.py > ...

```python
import numpy as np
import random

class RLAgent:
    def __init__(self, num_states, num_actions):
        self.q_table = np.zeros((num_states, num_actions))
        self.alpha = 0.1
        self.gamma = 0.9
        self.epsilon = 0.1

    def choose_action(self, state):
        if random.uniform(0,1) < self.epsilon:
            return random.randint(0, self.q_table.shape[1]-1)
        return np.argmax(self.q_table[state])

    def update(self, state, action, reward, next_state):
        best_next = np.max(self.q_table[next_state])

        self.q_table[state, action] += self.alpha * (
            reward + self.gamma * best_next - self.q_table[state, action]
        )
```

```
Epoch 30/30
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.6685 - loss: 0.9396 - val_accuracy: 0.6366 - val_loss: 1.1947 - learning_rate: 2.5000e-04

--- Generating Checkpoint Artifacts ---
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
Artifacts successfully saved to: c:\Users\Admin\Downloads\traffic-sign-classifier-main-20260321T060516Z-1-001\traffic-sign-classifier-main\docs\screenshots

--- To Explain ---
Sample 0 | Detected: 40 | Action: BRAKE
   ↳ Rationale: Vehicle speed 90 exceeds limit 40.
Sample 1 | Detected: 70 | Action: BRAKE
   ↳ Rationale: Vehicle speed 90 exceeds limit 70.
Sample 2 | Detected: 80 | Action: BRAKE
   ↳ Rationale: Vehicle speed 90 exceeds limit 80.
Sample 3 | Detected: 20 | Action: BRAKE
   ↳ Rationale: Vehicle speed 90 exceeds limit 20.
Sample 4 | Detected: 80 | Action: BRAKE
   ↳ Rationale: Vehicle speed 90 exceeds limit 80.

Checkpoint Complete.
```

- **Draft Model Card sections started; Ethics statement updated.**

Model Card progress work:

-classifier

Actions | Projects | Wiki | Security | Insights | Settings

traffic-sign-classifier / docs / model_card.md

sheena-magpantay Update model_card.md

Preview | Code | Blame | 73 lines (41 loc) · 1.77 KB

MODEL CARD-TRAFFIC SIGN CLASSIFIER

Model Details

- A Traffic Sign Classification model based on a Convolutional Neural Network (CNN) architecture.
- Designed to classify road and traffic signs from image inputs.
- Built with image preprocessing powered by OpenCV.
- Developed as an academic project.

Intended Use

This model is developed strictly for academic and educational purposes.

Intended for:

- Demonstrating applications of Convolutional Neural Networks (CNNs) in Computer Vision
- Learning image classification techniques using traffic sign datasets
- Experimentation and research in machine learning workflows

Not intended for:

- Real-world deployment in autonomous driving systems

Updated Ethics statement:

traffic-sign-classifier / docs / ethics_statement.md

sheena-magpantay Create ethics_statement.md

Preview | Code | Blame | 89 lines (70 loc) · 2.84 KB

Ethics, Policy, License, and Disclaimer

Overview

This document outlines the ethical considerations, intended use, limitations, risk mitigation strategies, license, and disclaimer for the Traffic Sign Classifier system.

The project focuses on the responsible development and evaluation of artificial intelligence models using Convolutional Neural Networks (CNN) and reinforcement learning techniques. The system is intended strictly for academic and research purposes.

Intended Use

The Traffic Sign Classifier is designed to:

- Demonstrate the application of Convolutional Neural Networks (CNN) in image classification
- Support academic learning and experimentation in computer vision
- Serve as a prototype model for traffic sign recognition

This system is **not intended for deployment in real-world or safety-critical environments**, such as autonomous driving systems, without extensive further development and validation.

Dataset Scope and Limitations

The dataset used in this study includes a limited set of traffic sign categories:

- Traffic lights
- Speed limit signs
- Stop signs